

Standardvärden för funktioner och metoder

C++ är väldigt flexibelt med hur parametrar kan ges till metoder. Förutom normal polymorfism (se [avsnittet Polymorfism](#)) kan man använda sig av standardvärden för parametrar till funktioner, konstruktörer och metoder. Med standardvärden avses att en eller flera parametrar har ett fördefinierat värde som används ifall anropet inte gett med just den parametern. Ett exempel förklarar saken bättre. Vi kan anta att vi behöver en klass för att lagra en viss färg. Färger brukar vanligen enkodas som ett separat värde på rött, grönt och blått. Man pratar om *RGB-värden*. Mängden av dessa tre komponenter bestämmer den slutliga färgens värde. För att få t.ex. röd färg sätts färgens röda komponent till full intensitet, medan gröna och blå setts till 0. För att få vitt sätts alla komponenter till fullt och tvärtom för svart. Vi kan anta i vårt exempel att intensiteten hos en komponent är mellan 0 och 1.0 (inklusive). 1.0 är full intensitet. Vi kan då skriva en enkel klass för en färg:

```
class Color {
public:
    // konstruktor
    Color (float Red = 0.0, float Blue = 0.0, float Green = 0.0);
    Color (const Color & Original);

    // accessera de enskilda komponenterna
    float red () const { return m_Red; }
    float green () const { return m_Green; }
    float blue () const { return m_Blue; }
    void setRed (float Red) { m_Red = Red; }
    void setGreen (float Green) { m_Green = Green; }
    void setBlue (float Blue) { m_Blue = Blue; }

private:
    // de enskilda komponenterna
    float m_Red;
    float m_Green;
    float m_Blue;
};

Color::Color (float Red, float Blue, float Green) {
    m_Red = Red;
    m_Green = Green;
    m_Blue = Blue;
}

Color::Color (const Color & Original) {
    m_Red = Original.red ();
    m_Green = Original.green ();
    m_Blue = Original.blue ();
}
```

Vi har i den ena konstruktorn använt oss av fördefinierade värden för de olika färgkomponenterna. Vi kan alltså lämna bort dessa parametrar om vi tycker de fördefinierade värdena är bra. Vi kan t.ex. skapa färger på följande sätt:

```
Color White ( 1, 1, 1 );    // full intensitet på alla komponenter blir vit
Color Red ( 1 );           // översätts till: Color Red ( 1, 0.0, 0.0 );
Color Black;               // översätts till: Color Black ( 0.0, 0.0, 0.0 );
```

Den regel som gäller för fördefinierade värde är att man kan lämna bort värde med start från *höger*. Vi kan t.ex. inte tro att objektet `Red(1)` ovan skulle bli t.ex. `Red(0.0, 0.0, 1)`, utan de värden som inte finns lämnas alltid bort från höger. Kompilatorn kan annars inte veta vilket värde man lämnat bort. Man kan i en definition göra endast några värden till fördefinierade:

```
void Image::fill (const Coordinate & UpLeft, const Coordinate & DownRight, const
```

Denna metod kan användas till att fylla en given rektangel i en bild med en viss färg. Här måste man ge parametrarna `UpLeft` och `DownRight`, medan `amn` kan lämna bort `FillColor` om man är nöjd med den fördefinierade (vit). Notera att även klasser kan vara fördefinierade värden.

Man måste alltid ge åtminstone så många värden till ett anrop som det finns parametrar utan fördefinierade värden. Det är inte heller tillåtet att två funktioner eller metoder "ser lika ut" ifall den ena t.ex. anropas utan värden. Följande är inte tillåtet:

```
class Color {  
public:  
    // konstruktor  
    Color (float Red = 0.0, float Blue = 0.0, float Green = 0.0);  
    Color ();  
    ...  
};
```

Kompilatorn vet inte vilken konstruktor som skall användas om man instantierar ett objekt som `Color NyFarg;`. Båda konstruktorerna kunde användas.

[Förutgående](#)
Polymorfism

[Hem](#)
[Upp](#)

[Nästa](#)
Statiska metoder och
medlemmar